

SAM for Life: Software Development Life Cycle (SDLC) & Technical Stack Documentation

1. Executive Summary

This document outlines the Software Development Life Cycle (SDLC) methodology, architectural design, and technical stack used to build the web platform for **SAM for Life**. This documentation is prepared for company submission and technical compliance auditing.

2. Technical Stack Specifications

The system is designed with a decoupled architecture, dividing responsibilities between a client-side React application and a server-side Node.js Express REST API.

2.1. Frontend Client

- **Base Library:** React 19 (Single Page Application architecture)
- **Client-Side Routing:** React Router Dom v7
- **CSS & Styling Framework:** Tailwind CSS & Tailwind CSS Animate (responsive grids, components, animations)
- **Dynamic Documents:**
- **PDF Generation:** `jsPDF` (generates donation receipts and invoices client-side)
- **Excel Data Exports:** `xlsx` (SheetJS, formats and downloads form submissions into Excel workbooks)
- **API Communication:** Axios (configured with interceptors to inject JWT security tokens)
- **Build System:** Craco (CRA Webpack wrapper)

2.2. Backend REST API

- **Runtime Environment:** Node.js
- **Application Framework:** Express
- **Database Driver:** Mongoose (MongoDB Object Document Mapper)
- **Email Engine:** Nodemailer (SMTP configuration pointing to IONOS mail servers)
- **Security & Auth:** `bcryptjs` (password hashing) and `jsonwebtoken` (session signing)
- **Payment Processor:** Stripe SDK (with a configurable local simulation engine for sandboxed testing)

3. Database Schema Models

The system runs on MongoDB. If the connection fails or is unconfigured, the server automatically switches to a local file-based database (`db.json`) to guarantee service availability.

3.1. SiteSettings (Singleton Database Configuration)

Stores all customizable website copy, official charity details, and email templates:

- `receipt\_charity\_number`, `receipt\_address`, `receipt\_thank\_you` (used for PDF generation)
- **Custom Email Templates:** Store HTML bodies and subject lines for public forms:
- `email\_contact\_admin\_subject`, `email\_contact\_user\_subject`, `email\_contact\_user\_body`
- `email\_volunteer\_admin\_subject`, `email\_volunteer\_user\_subject`, `email\_volunteer\_user\_body`
- `email\_partnership\_admin\_subject`, `email\_partnership\_user\_subject`, `email\_partnership\_user\_body`
- `email\_fundraise\_admin\_subject`, `email\_fundraise\_user\_subject`, `email\_fundraise\_user\_body`
- `email\_donation\_admin\_subject`, `email\_donation\_user\_subject`, `email\_donation\_user\_body`

3.2. PaymentTransaction (Donation Records)

Tracks payment states, Stripe session IDs, and donor info:

- `session\_id` (String, Unique)
- `amount` (Number)
- `currency` (String, default: `gbp`)
- `frequency` (String: `one-time` or `monthly`)
- `donor\_name` / `donor\_email` (Strings)
- `payment\_status` (String: `initiated`, `paid`, etc.)
- `receipt\_sent` (Boolean, default: `false` - tracks whether receipts have been successfully emailed)

4. Software Development Life Cycle (SDLC) Process

The project followed a structured, iterative Software Development Life Cycle matching industrial Agile practices.

4.1. Phase 1: Requirements Analysis & System Planning

- **Objective:** Define features required for a donor-facing non-profit platform and its corresponding administration portal.
- **Scope Defined:**
 - Dynamic content management (CMS) for stories, program details, news, and site settings.
 - SMTP email triggers for form submissions (automatic auto-reply receipts for visitors, notifications to admins).
 - Secure donation checkout with card payments.
 - In-browser receipt and invoice downloads.
 - Fast load times on mobile networks.

4.2. Phase 2: System Architecture Design

- **Frontend:** A responsive Single Page Application. We selected `jsPDF` for client-side PDF rendering to avoid heavy server-side image processing.
- **Backend:** Express API structure. Database operations were wrapped in a singleton Mongoose database helper, featuring a fallback JSON database for zero-dependency local setups.
- **Security Model:** Enforces HTTPS, hashes admin passwords with salt factors (bcrypt), and secures administrative routes using JSON Web Tokens (JWT) sent via HTTP Bearer headers.

4.3. Phase 3: Implementation & Development

- **Database & Seeding:** Created schema definitions and structured seed migrations that populate initial default website texts and SMTP templates on database startup.
- **SMTP Notifications:** Wrote a unified mailer module that compiles HTML containers and replaces customizable variables (e.g. `{name}`, `{amount}`) on form submissions.
- **Excel Data Exports:** Installed SheetJS (`.xlsx`) and created a utility that gathers user inbox logs, partitions them by database collections, and formats them into a multi-sheet spreadsheet.
- **Speed Optimizations:**
 - Implemented query parallelization in `/api/cms/all` using `Promise.all()`.
 - Added CDN Edge caching headers (`Cache-Control: s-maxage=10, stale-while-revalidate=3600`).
 - Added frontend LocalStorage caching using the Stale-While-Revalidate (SWR) rendering pattern.
 - Implemented dynamic Unsplash image compression and lazy loading.

4.4. Phase 4: Testing & Quality Assurance

- **Automatic Compilation Checks:** Validated frontend Webpack compiler builds and Express server start logs.
- **Payment & Webhook Verification:** Simulated transaction checkouts using mock sessions, confirming that the backend correctly handles payments and triggers emails without duplicating them.
- **Cross-Origin Resource Sharing (CORS):** Validated frontend/backend connectivity locally and in simulated production environments.

4.5. Phase 5: Deployment & Environment Configuration

- **Frontend Deployment:** Static web hosting on Vercel Edge Networks, linked with the custom domain `samforlife.org` (and automatic SSL configuration).
- **Backend Deployment:** Serverless Node.js functions on Vercel, connected to a MongoDB Atlas cluster hosted on AWS.
- **Alternative Database Setup:** Configured step-by-step setup guides to host secure, password-protected MongoDB databases on an AWS EC2 instance with custom EBS storage sizes under the AWS Free Tier.